应用说明

神经网络处理单元（NPU）指南

Pranav Siddappa, Nima Eskandari, Nikhil Dasan, Tushar Sharma, Adithya Thonse

摘要

本《神经网络处理单元指南》为在 F28P55x NPU 上部署机器学习解决方案提供了全面框架，尤其面向汽车和工业应用。借助这款片上硬件加速器，C2000™Ware 用户可实现实时推理，用于预测性维护、异常检测、传感器融合和先进控制系统，同时满足这些领域对确定性性能的要求。通过一个实用的正弦函数近似示例，

本指南将带领工程师完成从架构设计到硬件验证的整个流程
，并展示 NPU 在内存和计算资源受限情况下的能力。文档介绍了有效使用 NPU 所需的量化技术、TI 工具链编译流程，以及在 CCS 工程中的集成策略。汽车和工业领域的开发人员可从中获得实用知识，用于开发高效的嵌入式机器学习应用，以满足特定环境对时序、功耗和可靠性的严格要求。

目录

1 引言 3
1.1 NPU 的定义与用途 3
1.2 主要能力 3
1.3 技术限制 3
2 开发流程概述 4
2.1 模型开发阶段 4
2.2 模型编译阶段 4
2.3 应用集成阶段 4
3 示例模型创建（Python） 5
3.1 模型选择依据 5
3.2 模型架构设计 5
3.3 训练详情 6
4 嵌入式平台量化 9
4.1 量化方法：QAT 与 PTQ 9
4.2 量化框架和封装模块 10
5 模型验证 12
5.1 两阶段训练策略 12
5.2 训练阶段对比 12
5.3 验证结果与指标 12
6 模型测试 14
6.1 推理设置与方法 14
6.2 测试结果与可视化分析 14
6.3 定量性能指标 15
7 将模型迁移至 TI MCU（C2000 – F28P55x）[初学者级别] 16
8 将模型迁移至 TI MCU（C2000 – F28P55x）[开发者级别] 17
8.1 编译前提条件 17
8.2 配置文件设置 17
8.3 编译流程 20
9 设置 MCU 工程 21
9.1 为 NPU 应用创建 CCS 工程 21
9.2 了解 NPU 接口 23
10 在嵌入式环境中测试模型 25
10.1 可视化性能评估 25

版权所有 © 2026, Texas Instruments Incorporated

10.2 定量性能指标 25
11 NPU 在实时信号链中的集成 27
11.1 应用框图 27
11.2 应用程序代码实现 27
11.3 使用的硬件组件 28
11.4 硬件验证结果 28
12 关键设计决策及其影响 30
12.1 NPU 对数值的处理 30
12.2 支持的神经网络层及限制 30
12.3 模型复杂度和大小限制 31
13 基准测试 32
13.1 模型性能对比 32
13.2 性能分析 33
13.3 流水线阶段计时测量 34
14 总结 35
14.1 主要能力与限制 35
14.2 开发工作流程 35
14.3 模型设计注意事项 35
14.4 实现挑战与解决方案 35
14.5 更广泛的应用 35
15 参考文献 37

商标

C2000™ 是 Texas Instruments 的商标。
PyTorch® 是 Linux Foundation 的注册商标。
所有商标均归各自所有者所有。

版权所有 © 2026, Texas Instruments Incorporated

1 引言

1.1 NPU 的定义和用途

F28P55x 神经网络处理单元（NPU）是集成在 TI C2000 微控制器架构中的专用硬件加速器，专为高效执行神经网络计算而设计。这种专用芯片使机器学习推理可直接在嵌入式器件上运行，无需外接处理器或连接云端。作为 C2000 生态系统的重要组成部分，NPU 与主 CPU、模拟前端和控制外设协同工作，为实时控制系统提供复杂的智能处理能力。NPU 体现了 TI 将先进机器学习能力带入资源受限环境的投入，而这类环境对确定性性能尤为重视。

1.2 主要能力

F28P55x NPU 为 C2000 应用提供了多项重要能力：

- *Accelerated Neural Network Inference:*硬件优化的神经网络运算执行能力显著优于主 CPU 上的软件实现；根据模型架构不同，典型加速比最高可达 70x。
- *Integer-Based Computation:*针对定点运算进行了专门优化，可在嵌入式资源约束下实现高能效推理。
- *Real-Time Processing:*执行过程具有确定性，可满足汽车和工业应用中控制系统对可预测时序的要求。
- *Peripheral Integration:*可与 ADC、DAC 及其他 C2000 外设无缝协同，支持完整的信号处理和控制工作流。
- *Parallel Operation:*主 CPU 执行其他任务时，NPU 仍可进行神经网络计算，从而提高系统吞吐量。
- *Automotive/Industrial Focus:*其设计满足这些严苛应用领域对可靠性、温度范围和长期供货能力的严格要求。

1.3 技术限制

F28P55x NPU 功能强大，但仍受若干约束，这些约束会影响应用设计：

- *Architectural Limitations:*与 LSTM 或 Transformer 等复杂架构相比，NPU 对 CNN、MLP 等采用 ReLU 激活函数的神经网络拓扑支持更好。
- *Precision Tradeoffs:*NPU 执行需要量化，这会相较浮点实现造成精度损失，因此需要谨慎选择训练方法以保持准确率。
- *Development Workflow Complexity:*模型编译和部署需要特定工具链，因此相比标准微控制器编程会增加开发步骤。

这些能力与限制共同界定了 F28P55x NPU 在汽车和工业嵌入式系统中的实际应用范围；成功实现应用的关键，是在计算能力与资源约束之间取得平衡。

版权所有 © 2026, Texas Instruments Incorporated

2 开发流程概述

F28P55x NPU 应用的开发工作流按结构化流程展开，将机器学习模型开发与嵌入式系统部署衔接起来。本节概述完整开发周期，后续章节将进一步详细介绍各个环节。

2.1 模型开发阶段

NPU 开发首先需要设计和训练适合嵌入式部署的模型：

- *Model Architecture Design:*创建神经网络架构，在应用需求与 NPU 硬件限制之间取得平衡；通常应优先选择规模较小且层类型经过优化的网络。
- *Dataset Preparation:*整理具有代表性的训练数据，覆盖部署时预期的完整运行范围，并特别注意选择与量化兼容的输入归一化策略。
- *Quantization-Aware Training:*设计训练流程时，应将量化影响纳入训练过程，使模型学到的参数能够适应 NPU 仅支持整数运算的限制。
- *Model Validation:*使用与目标应用相关的指标验证模型性能，并在量化环境中同时评估准确率和计算效率。

2.2 模型编译阶段

模型训练完成后，必须将其转换为兼容 F28P55x NPU 的格式：

- *ONNX Export:*将训练后的模型转换为开放神经网络交换（ONNX）格式，作为编译工具链使用的互操作标准。
- *TI NPU Compiler Configuration:*通过配置文件设置编译参数，指定目标器件、优化策略和输入/输出要求。
- *Compilation Execution:*使用 TI 神经网络编译器（TI MCU NNC）处理 ONNX 模型，生成兼容 NPU 硬件的 C/C++ 代码产物。
- *Compilation Output Verification:*检查生成的头文件和库，确保转换正确，尤其要关注回归任务中的反量化等关键环节。

2.3 应用集成阶段

最后一个阶段是将编译后的模型集成到完整的嵌入式应用程序中：

- *CCS Project Setup:*创建 Code Composer Studio 工程，将生成的模型产物与应用专用代码一并纳入工程。
- *Hardware Peripheral Configuration:*配置必要外设（ADC、DAC 和通信接口），为神经网络提供输入并处理其输出。
- *Application Logic Implementation:*编写应用程序主逻辑，协调外设与 NPU 之间的数据流，并正确缩放输入和输出。
- *Hardware Testing:*在真实 F28P55x 硬件上、使用符合实际的运行条件验证端到端功能。
- *Performance Optimization:*微调应用程序，使推理速度、准确率和功耗之间达到最佳平衡。
- *Deployment Packaging:*为在汽车或工业环境中进行量产部署准备最终固件包。

版权所有 © 2026, Texas Instruments Incorporated

3 使用 Python 创建示例模型

本节通过创建一个正弦函数近似模型，展示 NPU 开发原则的实际应用。该示例提供了一个参考实现，涵盖从模型设计到部署至 F28P55x NPU 的完整工作流程。

3.1 模型选择依据

选择正弦函数作为演示示例，主要基于以下几方面考虑：

- *Mathematical Complexity:*正弦函数看似简单，实际代表一种非线性数学关系；对其建模需要神经网络能力，不能仅依靠简单线性近似。
- *Bounded Output Range:*正弦函数的输出范围限定在 -1 到 +1，边界明确，适合应用量化策略。
- *Visual Verification:*本示例使用示波器波形可视化（适用于正弦函数）。其他应用则应根据具体任务选择合适的验证方法，例如分类任务使用混淆矩阵、异常检测使用热力图，其他回归任务使用误差分布图。这些回归任务常见于汽车和工业领域的控制系统、信号处理及运动控制应用。
- *Complete Pipeline Demonstration:*正弦函数可用于演示完整的 模拟→数字→模 信号处理链路，其中 NPU 是核心处理单元。

3.2 模型架构设计

正弦函数近似模型采用多层感知机（MLP），其架构专门针对 F28P55x NPU 的限制进行了精简设计：

```
SineApproximator(
  (regressor): Sequential (
    (0): Linear(in_features=1, out_features=64, bias=True)
    (1): ReLU(inplace=True)
    (2): Linear(in_features=64, out_features=64, bias=True)
    (3): ReLU(inplace=True)
    (4): Linear(in_features=64, out_features=1, bias=True)))
```

Code 1: Sine Approximator Backbone Architecture for Sine_64_Model

该架构包含多项针对 NPU 部署作出的关键设计决策：

- *Input Layer:*单神经元输入，接收角度值（映射到 0 到 2π 弧度）。
- *Hidden Layers:*设置 2 个隐藏层，每层 64 个神经元，在准确率和资源效率之间取得平衡；NPU 可容纳每层最多 128 个神经元的模型。
- *Activation Functions:*选择 ReLU 激活函数，以提高计算效率并便于量化。
- *Output Layer:* 单神经元输出，给出预测的正弦值（范围为 -1 到 +1）。

版权所有 © 2026, Texas Instruments Incorporated

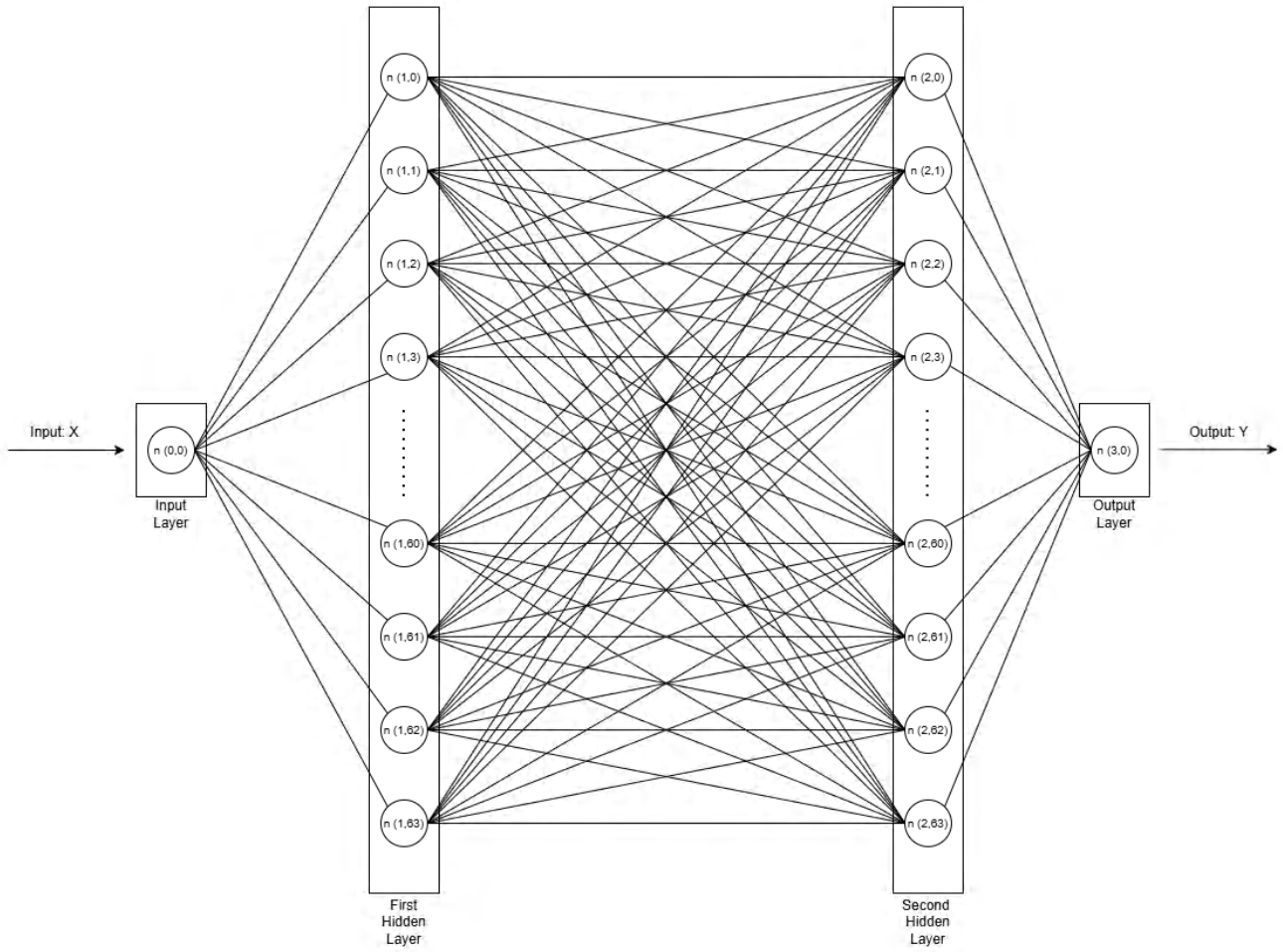


图 3-1. Sine_64_Model 的正弦近似器骨干架构

模型架构的规模根据 F28P55x NPU 的存储器和计算能力精心确定。关键限制是参数总数，而不是每层神经元数量的固定上限。本架构的各层规模为 1×64、64×64 和 64×1，共有 4,353 个参数，可在准确率与资源利用率之间取得平衡。器件可以容纳每层最多 128 个神经元的模型，但本例基于准确率和延迟因素选择了 64 个神经元。对于未使用特征提取器的较大网络（例如每层包含 512 或 1024 个神经元），其规模会超过 NPU 可用存储器。设计此平台的模型时，开发人员应重点关注参数总量。边缘 AI 实现普遍采用这种资源受限的模型设计方法；由于存储器、处理能力和能耗预算有限，必须仔细优化神经网络架构，才能在现有硬件条件下获得尽可能好的性能。

3.3 训练详情

模型训练按以下结构化流程进行：

3.3.1 Development Environment Setup

- *Example Model Setup:*
  – 使用 PyTorch® 框架设计和训练神经网络。– 使用 TINPUTinyMLQATFxModule 封装模块进行量化感知训练（量化详情见第 4 节）。– 使用辅助数值计算和可视化的库。

- *Generic User Setup:*
  – 为目标硬件安装 TI 模型优化工具。– 确保所选框架版本与 TI 编译工具链兼容。

版权所有 © 2026, Texas Instruments Incorporated

– 使用虚拟环境（conda、venv）配置一致的软件环境，以避免依赖冲突。– 根据应用领域加入相应的可视化库（例如用于传感器的信号绘图或用于计算机视觉的图像显示）。

3.3.2 Dataset Generation

- *Example Model Dataset Details:*
  – 随机生成 100,000 个角度值，范围为 0 到 2π 弧度。– 计算对应的正弦值作为训练目标。– 将数据划分为训练集和测试集，分别占 80% 和 20%。

- *Generic User Approach:*
  – 生成或采集覆盖完整输入范围的数据，以保持模型性能稳定。– 对于传感器应用，应在所有运行条件和环境变量范围内采集数据。– 对数据集进行平衡处理，避免模型训练产生偏差（分类任务尤其需要注意）。– 根据部署场景的预期输入范围采用适当的归一化方法。– 对分类任务使用分层抽样，以保持训练集和测试集中的类别分布。– 考虑在训练数据中添加可控噪声，以提高模型在实际部署中的鲁棒性。– 对于时序数据，在数据集准备过程中应妥善处理序列。

3.3.3 Model Training Configuration

- *Example Model Training Parameters:*
  – *Batch size:* 512 个样本 – *Learning rate:* 1e-5（刻意采用较小值以实现稳定收敛）– *Optimizer:* Adam – *Loss function:* 均方误差（MSE）– *Training epochs:* 160

- *Generic User Training Flow:*
  – 根据模型复杂度和可用存储器调整批量大小（存储器受限时应减小批量）。– 根据模型规模和收敛行为选择适当的学习率。– *Choose optimizer based on your task requirements:*

    - Adam，适用于通用训练并可加快收敛
    - SGD 配合动量，有时可获得更好的泛化能力
    - RMSprop，适用于循环神经网络 –

  Select loss function appropriate to your task:

    - 回归问题使用均方误差（MSE）
    - 分类任务使用交叉熵
    - 针对专用应用设计自定义损失函数。– 采用学习率调度以提高训练稳定性和最终模型准确率。– 考虑使用提前停止，以防止过拟合，尤其是在训练数据有限时。– 同时监测训练指标和验证指标，以确认模型具有良好的泛化能力。

3.3.4 Quantization-Aware Training Process

- *Example Model Quantization Process:*
  – 初始模型由 TINPUTinyMLQATFxModule 封装，以模拟量化效果。– 前向传播过程包含权重和激活值的量化仿真。– 反向传播时，梯度计算会考虑量化影响。– 定期验证量化模型的性能。

```
model = TINPUTinyMLQATFxModule(     model,     to
tal_epochs=(int)(MAX_EPOCH/10),     output_int=Fa
lse,     quantization_weight_bitwidth=2)
```

Code 2: Wrap model with quantization wrapper for fine tunning

- *Generic User Quantization Flow:*
  – *Select quantization approach:*

版权所有 © 2026, Texas Instruments Incorporated

- 采用 QAT 可提高准确率（但训练时间较长）
- 采用 PTQ 可加快开发（但可能牺牲准确率）

– *Configure for target hardware:*
- 使用适用于特定硬件的封装模块（例如 TINPUTinyMLQATFxModule）
- 设置适当的位宽（2/4/8 bits）和精度参数
- 设置微调所使用的训练轮数

– *Optimize training process:*
- 先使用浮点数进行初始训练，再通过量化进行微调
- 与浮点基准进行对比，以评估准确率影响

– *Mitigate quantization issues:*
- 监测激活值范围，避免发生截断
- 确保回归输出经过正确的反量化处理
- 使用权重/激活值直方图识别分布问题

版权所有 © 2026, Texas Instruments Incorporated

4 面向嵌入式平台的量化

神经网络量化是将权重和激活值的高精度浮点表示转换为较低精度格式（通常为整数）的过程。对于 F28P55x NPU，这种转换不仅是优化手段，更是基本要求，因为该硬件专为整数运算而设计。

对于本指南中的正弦函数近似模型，量化会将连续的数学关系转换为 NPU 可高效处理的形式，同时保留正弦波的关键特征。

4.1 量化方法：QAT 与 PTQ

神经网络量化主要有两种方法：训练后量化（PTQ）和量化感知训练（QAT）。了解二者的差异，对于为 F28P55x NPU 部署选择合适策略至关重要。

4.1.1 训练后量化（PTQ）

PTQ 对已经以浮点精度训练完成的模型进行量化。该方法较为简单，但可能损失准确率，尤其对于较小模型或对精度敏感的任务。

主要特点：

- 流程：正常训练模型 → 校准量化参数 → 转换为量化格式。
- 校准：使用具有代表性的数据集确定缩放因子。
- 开发速度：开发周期更短（无需重新训练）。
- 对精度的影响：通常比 QAT 带来更大的准确率损失。

优点：

- 对于已有训练模型，工作流程更简单。
- 无需修改训练流程。
- 部署路径更快。
- 开发阶段的计算需求较低。

局限性：

- 可能导致准确率明显下降。
- 对量化影响的控制较少。
- 对于本例中的正弦函数等回归任务，采用 PTQ 尤其具有挑战性。
- 补偿量化伪影的能力有限。

4.1.2 量化感知训练（QAT）

QAT 会在训练过程中纳入量化影响，使网络能够学到在量化条件下表现最佳的参数。这种方法通常能更好地保持准确率，但需要投入更多开发工作。

主要特点：

- 流程：初始训练 → 插入量化运算 → 继续训练 → 转换为量化格式。
- 仿真：在前向传播和反向传播过程中模拟量化影响。
- 开发速度：开发周期较长（需要额外训练）。

优点：

- 更好地保持模型准确率。
- 网络能够学习补偿量化影响。
- 对于精度敏感型应用尤其有价值。
- 在量化硬件上的性能更可预测。

局限性：

- 开发工作流程更复杂。
- 需要额外的训练计算资源。
- 延长产品部署所需时间。

版权所有 © 2026, Texas Instruments Incorporated

- 需要仔细调整超参数。

本例中的正弦函数近似模型选择了 QAT，因为它能更好地保持回归任务的准确率；这一点对于在整个输入范围内保持正弦波特征平滑至关重要。

4.2 量化框架和封装模块

TI 工具链提供了专用封装模块，可针对不同部署目标封装各类量化方法。了解这些封装模块，对于为应用选择合适的量化策略至关重要。

TI 的量化框架提供 4 种不同的封装模块，这些模块由以下两个关键维度组合而成：

- *Target Hardware:*
  – *Generic:* 针对 CPU 执行进行了优化（采用标准整数运算）。– *TINPU:* 专为 TI 神经网络处理单元硬件优化。
- *Quantization Approach:*
  – *QAT (Quantization-Aware Training):*在训练期间纳入量化影响。– *PTQ (Post-Training Quantization):* 在训练完成后应用量化。

由此形成 4 种封装模块选项：

表 4-1. 量化封装模块

Target/Approach	QAT	PTQ
Generic CPU	GenericTinyMLQATFxModule	GenericTinyMLPTQFxModule
TI NPU	TINPUTinyMLQATFxModule	TINPUTinyMLPTQFxModule

4.2.1 Generic Wrappers for CPU Quantization

Generic 封装模块（GenericTinyMLQATFxModule 和 GenericTinyMLPTQFxModule）面向 CPU 执行，并使用 PyTorch 原生量化 API：

```
from tinyml_torchmodelopt.quantization import GenericTinyMLQATFxModule # or from tinyml_torch
modelopt.quantization import GenericTinyMLPTQFxModule
```

Code 3: Generic Quantization APIs

这些封装模块适用于：

- 在没有 NPU 加速的 C2000/ARM 微控制器上部署。
- 在进行 NPU 专项优化前测试量化影响。
- 性能要求不太严格的应用。
- 模型原型开发和初步验证。

Generic 封装模块使用标准 PyTorch 量化 API，同时通过统一接口简化应用，可最大限度减少对现有模型代码的修改。

4.2.2 TINPU Wrappers for NPU Hardware Acceleration

TINPU 封装模块（TINPUTinyMLQATFxModule 和 TINPUTinyMLPTQFxModule）专为 TI 神经网络处理单元硬件加速器设计：

```
from tinyml_torchmodelopt.quantization import TINPUTinyMLQATFxModule # or from tinyml_torc
hmodelopt.quantization import TINPUTinyMLPTQFxModule
```

Code 4: TI NPU Specific Quantization APIs

这些封装模块适用于：

- 在 F28P55x 及其他采用 NPU 加速的 TI 器件上部署。
- 充分发挥 TI 硬件的性能。
- 需要实时推理的应用。

版权所有 © 2026, Texas Instruments Incorporated

- 保持与 NPU 编译工具的兼容性。

TINPU 封装模块纳入了 TI NPU 硬件的特定约束，使模型不仅能受益于通用量化，还能针对 NPU 架构进行专门优化。

版权所有 © 2026, Texas Instruments Incorporated

5 模型验证

正弦函数近似模型针对 F28P55x NPU 进行了两阶段验证，以优化其硬件性能。本节介绍验证方法、量化感知训练方法以及相关性能指标，用于展示模型的

有效性。

5.1 两阶段训练策略

训练过程采用精心设计的两阶段方法，以充分发挥受量化限制的 NPU 硬件性能：

5.1.1 Initial Training Phase

- 使用常规浮点权重训练模型。
- 重点学习底层正弦函数的数学规律。
- 建立基础的模式识别能力。
- 在引入量化限制前打下坚实基础。

5.1.2 Fine-Tuning Phase

- 使用针对 F28P55x 量化的权重训练模型。
- 模拟 NPU 实际执行的纯整数运算。
- 专门针对量化执行优化权重。
- *Quant_epoch*通常设置为 *max(5,float_epoch/10)*。

这种策略结合了两种方法的优势：先以完整浮点精度进行训练，建立稳健的特征提取能力；再针对量化权重进行微调，优化模型在目标硬件上的部署性能。

5.2 训练阶段对比

表 5-1 对比了常规浮点训练最后一个训练轮次的性能指标与
量化感知微调结束后最后一个训练轮次的性能指标：

表 5-1. 量化前后的指标对比

Metric	Float Training metrics after 160 epochs	QAT Fine-tuning metrics after 16 epochs	Change (%)
Validation Loss	0.00181	0.00105	Improved by 42%
Validation MAE	0.02031	0.0215	Degraded by 6%
Validation R^2 Score	0.9963	0.9989	Improved by 0.26%

标准浮点模型在常规环境中表现出色，但受 NPU 仅支持整数运算的限制时，准确率会大幅下降。QAT 会在整个训练过程中模拟量化影响，使神经网络能够相应调整参数。这种优化可确保模型部署到 F28P55x NPU 后，在利用其专用神经网络加速能力的同时，仍能保持计算可靠性

能力。

验证损失（-42.0%）和 R^2 分数（+0.26%）的改善表明，QAT 能够提升特定性能指标，同时使模型适应 NPU 部署。MAE 小幅增加（+5.9%）体现了量化过程固有的权衡。这些结果验证了量化感知训练方法能够帮助资源受限的嵌入式硬件获得最佳性能。

5.3 验证结果与指标

验证采用了简单的方法，以便客观评估性能：

- *Validation Dataset:*将生成数据中的 20% 留出，不用于训练。
- *Multiple Metrics:*使用相互补充的评估指标进行全面评估。
- *Quantization Simulation:*验证时采用与目标硬件相同的量化方案。

版权所有 © 2026, Texas Instruments Incorporated

经过量化感知训练的 Sine_64 模型取得了良好的性能指标。考虑到它是一个简单的机器学习模型，这些结果符合预期，并证明该模型适合部署到 NPU：

- 验证损失通过均方误差（MSE）衡量。均方误差是预测值（ ¡）与实际值（y_i）之差的平方的平均值。数值越低，模型性能越好。

$$MSE = 1/n\sum_0^n(\mathrm{yi} - \hat{\mathrm{y}}\mathrm{i})^2 \quad (1)$$

- 平均绝对误差用于衡量预测值与实际值之差的绝对值平均数。与 MSE 不同，该指标不对误差平方处理，因此对离群值不太敏感。

$$MAE = 1/n\sum_0^n|\mathrm{yi} - \hat{\mathrm{y}}\mathrm{i}| \quad (2)$$

- R^2 表示与使用均值作为预测值相比，模型对数据方差的解释程度。取值范围为 0 到 1，1 表示预测完全准确。验证集 R^2 分数表示模型对给定数据方差的解释程度。

$$\mathrm{R}^2 = 1 - \left(\sum_0^n(\mathrm{yi} - \hat{\mathrm{y}}\mathrm{i})^2\right)/\left(\sum_0^n(\mathrm{yi} - \bar{\mathrm{y}})^2\right) \quad (3)$$

变量：

- yi = 验证数据集中第 i^{th} 个样本的实际（真实）正弦值
- i = 神经网络预测的第 i^{th} 个样本的正弦值
- = 验证数据集中所有实际正弦值的均值

表 5-2. 训练与验证指标

指标	指标	含义
验证损失	0.00038	均方误差极低，表明预测精度较高。
验证 MAE	0.01305	在正弦值范围（-1 至 +1）内，平均绝对误差约为 1.3%。
验证 R^2 分数	0.9993	决定系数接近 1，表明模型能够解释正弦值中 99.93% 的方差。

这些指标体现了以下关键成果：

- 高精度：尽管受到量化限制，模型仍实现了低于 1% 的误差。
- 性能稳定：较高的 R^2 分数表明模型在整个输入范围内都能可靠地进行预测。
- 抗量化能力：即使仅执行整数运算，模型性能下降仍很小。
- 适合部署：这些指标验证了该模型适用于 NPU 实现。

版权所有 © 2026, Texas Instruments Incorporated

6 模型测试

成功完成训练和验证后，还必须全面测试正弦函数近似模型，以确认其在实际应用中的性能。本节介绍用于评估模型是否适合部署到 NPU 的测试方法、推理流程和结果评估技术。

6.1 推理设置与方法

本示例使用专用推理 notebook 和 ONNX Runtime，在模型部署到硬件前进行验证。该框架实现了封装式测试类，既提供简洁的预测接口，也保持一致的数据处理方式。

测试方法按以下步骤进行：

- *Model Loading:*使用 ONNX Runtime 加载 ONNX 模型，以便访问同一个模型；该模型最终也会编译后部署到 NPU。
- *Input Preparation:*在整个输入范围 0 到 2π 弧度内生成测试点。
- *Reference Comparison:*将每个预测结果与数学函数计算出的正弦值进行比较。

此测试方法用于确认导出的 ONNX 模型（特别是包含反量化层的版本）能够在整个范围内正确近似正弦函数，作为开始
硬件实现前的关键质量门槛。

6.1.1 Generic User Testing Approach

- *Model Validation:*使用适当的框架测试导出的模型。
- *Representative Test Data:*创建能够反映部署条件和边缘情况的测试数据集。
- *Domain-Specific Metrics:*根据应用需求选择评估指标：– *Classification:* 准确率、精确率、召回率、F1 分数。– *Regression:* MAE、MSE、R^2 分数。– *Signal processing:* SNR、互相关、频率响应。– *Vision:* 目标检测的 IoU、mAP，以及图像质量的 SSIM。

- *Performance Benchmarking:*测量推理速度、存储器使用量和功耗。
- *Comparison Baselines:*在适用情况下，与参考算法进行基准比较。
- *Environmental Testing:*对于关键应用，应在不同温度范围、电压变化或其他环境因素下进行测试。

该验证是进入硬件实现前的重要质量门槛，可确保量化模型在投入硬件部署前满足应用要求。

6.2 测试结果与可视化分析

6.2.1 Visual Performance Assessment

测试的一个关键环节，是直观对比预测正弦值与实际正弦值。notebook 会生成一幅综合图表，在整个输入范围内将模型预测结果叠加到真实正弦函数曲线上。

通过该可视化结果，可以立即判断模型近似效果。在正弦函数示例中，
预测曲线与实际正弦曲线十分接近，肉眼几乎看不出偏差。

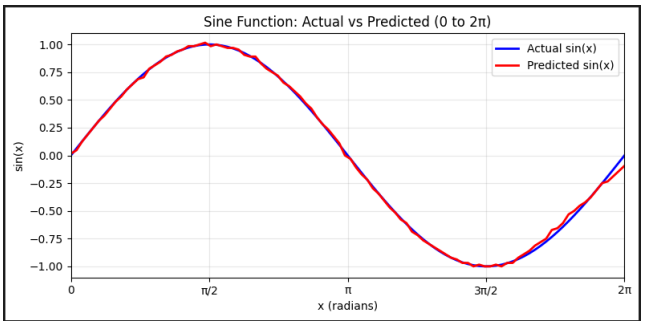


图 6-1. Python 中神经网络正弦函数近似性能

版权所有 © 2026, Texas Instruments Incorporated

图 6-1 展示了训练后的神经网络在整个输入范围内近似正弦函数的性能。蓝线表示真实的数学正弦函数，红线表示量化感知训练网络模型作为 ONNX 文件运行时生成的预测结果。该图展示了训练后的神经网络在整个输入范围内近似正弦函数的性能。

6.3 定量性能指标

除目视检查外，测试框架还会计算全面的误差指标，以精确量化预测准确率。测试期间得到的以下误差指标可用于准确评估模型性能：

- *Mean Absolute Error (MAE)*:0.013827
- *Maximum Error*:0.093750
- *Minimum Error*:0.000126

这些定量结果补充了图表提供的可视化评估，以数值方式确认模型在量化训练限制下仍具有出色的近似能力。结合可视化和数值验证，可在硬件实现前全面确认模型性能。

版权所有 © 2026, Texas Instruments Incorporated

7 将模型迁移到 TI MCU（C2000 – F28P55x）[初学者级别]

将 ONNX 模型部署到 F28P55x 可采用两种不同路径，以满足不同用户的需求和技术水平。

对于希望采用简洁、高效设计的用户，初学者路径提供了配置要求较少的简化工作流。这种方法抽象了许多技术细节，同时仍能实现可用的功能。

用户可参阅《TI Neural Network Compiler for MCUs User’ s Guide - 2.1.0.LTS》第 1、2 节，按其中的分步说明获取部署所需的已编译库和头文件。

版权所有 © 2026, Texas Instruments Incorporated

8 将模型迁移到 TI MCU（C2000 – F28P55x）[开发者级别]

开发者路径适合需要全面控制编译过程并深入集成现有系统的工程师。虽然步骤更多，但这种方法能够完整呈现部署流程的每个环节，也允许广泛自定义编译参数。以下各节将详细介绍该工作流，帮助开发者根据具体应用需求优化神经网络实现。

本节介绍将数学模型转换为硬件兼容文件的完整工作流，以便将这些文件集成到 Code Composer Studio 工程中。

8.1 编译前提条件

开始模型编译前，必须正确配置若干专用工具和环境。此准备阶段是成功完成编译和部署的关键。

8.1.1 Required TI Software Components

编译工具链依赖多个 TI 专用代码库和工具：

- *tinyml-modelmaker*
  – *Purpose:* 为模型开发和编译提供统一接口。– *Function:* 协调编译工作流并与其他组件集成。
- *tinyml- modeloptimization*
  – *Purpose:* 模型量化和优化工具包。– *Function:* 提供工具与封装模块，用于专门为 TI NPU 部署准备模型。
- *C2000Ware SDK*
  – *Purpose:* 为 C2000 微控制器提供平台专用驱动程序、库和工具。– *Function:* 为 F28P55x 平台提供必需的驱动程序和硬件抽象层。
- tinyml-modelzoo
  – *Purpose*：提供经过整理、针对嵌入式系统优化的神经网络模型集合。– 功能：通过 tinyml-modelmaker 方法编译模型时必需。

8.1.2 Environment Setup Process

编译环境按以下结构化流程搭建：

- *Clone Required Repositories:*
  – git clone https://github.com/TexasInstruments/tinyml-tensorlab/tree/main/tinyml-modelmaker – git clone https://github.com/TexasInstruments/tinyml-tensorlab/tree/main/tinyml-modeloptimization – git clone https://github.com/TexasInstruments/tinyml-tensorlab/tree/main/tinyml-modelzoo – *Run Initialization Script:*
  - *cd into **tinyml-modelmaker** repo*
  - 执行 *setup_all.sh*，完成整个环境的配置
  - 执行 *setup_c2000ware.sh*，配置 C2000Ware SDK
  - 执行 *setup_cg_tools.sh*，安装 TI 代码生成工具
- *Python Environment Setup:*
  – 建议使用专用 Python 虚拟环境，避免依赖冲突。– 所需的关键 Python 软件包包括：
  - ONNX 和 ONNX Runtime
  - TVM（TI MCU NNC）框架
  - NumPy 及相关数值计算库

8.2 配置文件设置

编译过程由 *config.yaml file* 控制，其中定义了将 ONNX 模型转换为 NPU 兼容代码所需的关键参数。这种配置方式既灵活，又能保持编译流程一致。

版权所有 © 2026, Texas Instruments Incorporated

8.2.1 Configuration File Structure

用于编译正弦函数模型的典型 config.yaml 文件包含以下主要部分：

```
common:  # The common section can be plainly copied as it is     target_module: 'timeseries'     task_type: 'generic_timeseries_regressio
n'     target_device: 'F28P55'     dataset:     dataset_name: Sine # Can be anything, used for directory name      enable: False      # Please
note the 'False'. Since model is already trained.  training: enable: False  # Please note the 'False'. Since model is already trained.     mod
el_name:  'SineModel_1e-05_160_64_Dequant'   # Can be anything, used for                                                directory name      outp
ut_int: False # We need the model to dequantize the values after                                 running on NPU and return float value compilation
:     enable: True     model_path: "path//to//SineModel_1e-05_160_64_Dequant.onnx"
```

Code 5: Example Configuration File

配置文件中的各部分用于指定编译过程的相应参数：

- *Common Settings:*指定基本任务类型和目标硬件。– *task_type* 标识机器学习任务类别。– *target_device* 启用 F28P55x 专用优化。
- *Dataset Settings:*控制数据集处理方式。– 使用预训练模型时通常禁用该项。
- *Training Settings:*配置训练参数。– 对于回归任务，*output_int:* 设置为 False 至关重要，可确保输出保留正确的浮点值。– 使用预训练模型时通常禁用训练。
- *Compilation Settings:*控制模型转换。– *enable:* 设置为 True 时启用编译阶段。– *model_path* 指定包含反量化层的 ONNX 模型位置。

8.2.1.1 需要反量化标志的模型

output_int: 设置为 False 对以下情况至关重要：

- *Regression Models:*预测连续值的模型，例如：– 时间序列预测（温度、压力、电压预测）– 控制系统建模（PID 系数估算）– 函数近似（如本例中的正弦函数）– 信号滤波应用– 传感器校准模型
- *Normalization-Heavy Models:*输出缩放具有重要意义的应用：– 输出归一化到特定范围的模型– 输出经过校准的传感器融合算法– 物理量估算（力、转矩等）
- *Multi-Output Models:*某些输出需要浮点精度的情况：– 分类/回归混合模型– 姿态估计（角度需要浮点数）– 坐标回归（物体定位）

如果未设置此标志，这些模型将生成量化整数输出，不适用于要求连续值范围或精确小数输出的应用。

8.2.2 Special Configuration for Regression Models

正弦函数近似等回归任务需要特殊处理，以便将 NPU 的纯整数输出正确反量化为浮点值。

版权所有 © 2026, Texas Instruments Incorporated

8.2.2.1 输出反量化标志

- 回归任务必须将 *output_int:* 设置为 False。
- 这样可指示编译器在生成的代码中保留反量化操作。
- 如果没有此标志，模型将输出整数，不适用于正弦函数近似。

8.2.2.2 修改编译器常量

除了配置文　　件之外，还需要修改编译器，以　　支持浮点输出。

- 定位并打开 *tinyml-modelmaker/ai_modules/timeseries/constant.py*。
- 在文件中向下滚动，找到 *COMPILATION_C28_HARD_TINPU* 字典：

```
COMPILATION_C28_HARD_TINPU= dict(      target= "c, ti-npu type=hard skip_normalize=true output_
int=true",      target_c_mcpu='c28',      cross_compiler=CL2000_CROSS_COMPILER,

)
```

Code 6: Reference Constant for Non- Regression Models

- 此现有常量针对分类任务配置，其中：– *type=hard* 指定使用硬件 NPU 加速（而非 CPU）。– *skip_normalize=true* 跳过归一化/缩放。– *output_int=true* 强制输出整数。

- 复制现有常量并进行关键修改，以创建名为 *COMPILATION_C28_HARD_TINPU_TEMP* 的新常量：

```
COMPILATION_C28_HARD_TINPU_TEMP = dict(      target= "c, ti-npu type=hard skip_normalize=false o
utput_int=false",      target_c_mcpu='c28',      cross_compiler=CL2000_CROSS_COMPILER,

)
```

Code 7: Extra Constant added to handle Regression Models

- 为适配回归模型，修改以下参数：
- *type=hard:* – 保持不变。该设置将编译目标指定为 NPU 硬件加速器，而不是 *soft*（该项会改由 CPU 执行神经网络计算）。
- *skip_normalize=false*：– 示例应用不需要归一化，因为它是一个简单的演示模型。

- *output_int=false:* – 编译器生成浮点输出而不是整数。– 保留回归任务所需的小数精度。– 支持表示完整的连续值范围。

- *target_c_mcpu='c28'*：保持不变——指定生成 C28x 架构的代码。
- *cross_compiler=CL2000_CROSS_COMPILER:*保持不变。定义要使用的编译器工具链。

8.2.2.3 更新编译字典

默认编译字典必须修改，以使用自定义常量：

- 将当前使用的常量从 *COMPILATION_C28_HARD_TINPU* 更改为 *COMPILATION_C28_HARD_TINPU_TEMP*。

```
TASK_TYPE_GENERIC_TS_REGRESSION: {
      COMPILATION_DEFAULT: dict(
-  compilation=dict(**COMPILATION_C28_HARD_TINPU,
   cross_compiler_options=CROSS_COMPILER_OPTIONS_F28P55, )
+  compilation=dict(**COMPILATION_C28_HARD_TINPU_TEMP,
```


版权所有 © 2026, Texas Instruments Incorporated

```
cross_compiler_options=CROSS_COMPILER_OPTIONS_F28P55, ) ),
```

代码 8：为适配回归模型所做的修改

8.3 编译流程

将 ONNX 模型转换为兼容 F28P55x NPU 的代码，需要经过 TI MCU 神经网络编译器（TI MCU NNC）执行的多个阶段。本节将逐步介绍编译工作流和内部转换过程。

8.3.1 启动编译

配置文件设置完成后，可通过 TinyML modelmaker 脚本启动编译：

```
cd tinyml-modelzoo /<path/to/run_tinyml_modelzoo.sh> <path/to/config_file>
```

代码 9：将 ONNX 文件编译为嵌入式兼容文件的命令

正弦函数示例使用以下命令：

```
./run_tinyml_modelzoo.sh   ./config.yaml
```

代码 10：启动编译的命令示例

该命令会启动完整的编译流程，将 ONNX 模型转换为针对 F28P55x NPU 优化的 C/C++ 代码。

8.3.2 编译阶段

编译器分 3 个主要阶段处理模型：

- 模型验证
  – 检查 ONNX 模型能否在 NPU 上运行。– 确认支持模型中的所有运算（例如本例正弦模型中的线性层和 ReLU）。– 验证模型是否符合 NPU 存储器限制。

- 模型转换
  – 将 ONNX 模型转换为针对 NPU 优化的格式。– 应用适用于整数运算的量化参数。– 对于本例中的正弦近似等回归模型，保留反量化信息。

- 代码生成
  – 创建可在 F28P55x 上运行的 C/C++ 代码。– 生成两个主要文件：
  - 头文件（tvmgen_default.h），包含函数声明
  - 库文件（mod.a），包含编译后的模型

8.3.3 常见问题

虽然编译器会处理大部分复杂工作，但应留意以下常见消息：

- 不支持的操作：模型包含 NPU 无法运行的运算。
- 超出内存限制：模型超出边缘器件的容量。
- 反量化错误：对于回归模型，可能需要检查输出反量化设置。

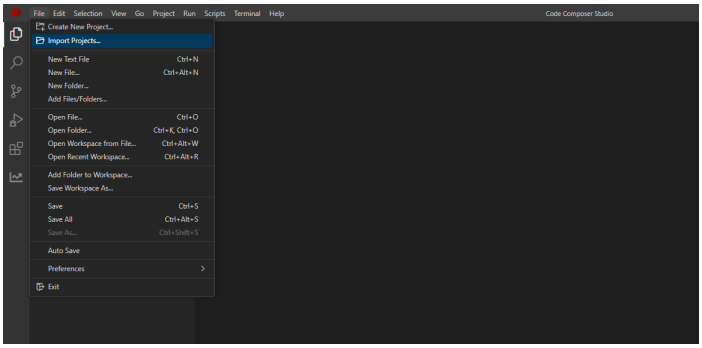
版权所有 © 2026, Texas Instruments Incorporated

9 配置 MCU 工程

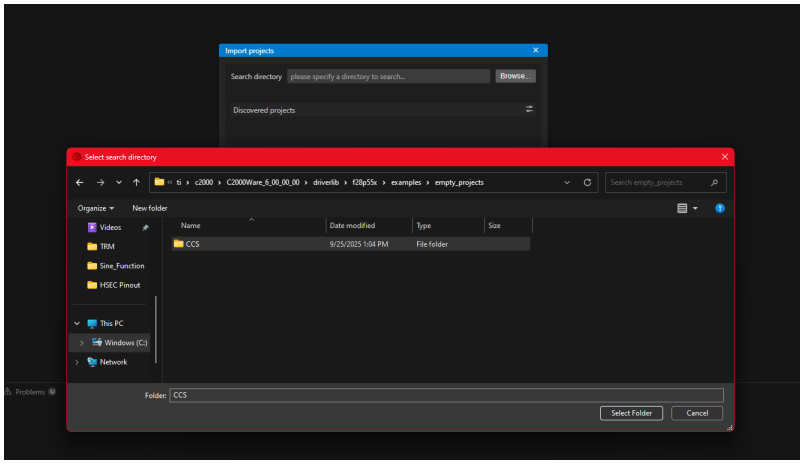
成功将神经网络模型编译为兼容 NPU 的文件后，下一关键阶段是将这些组件集成到 Code Composer Studio（CCS）工程中，以便在 F28P55x 微控制器上运行。本节指导工程师建立工程结构、配置开发环境，并实现利用 NPU 能力所需的应用程序框架。

9.1 为 NPU 应用创建 CCS 工程

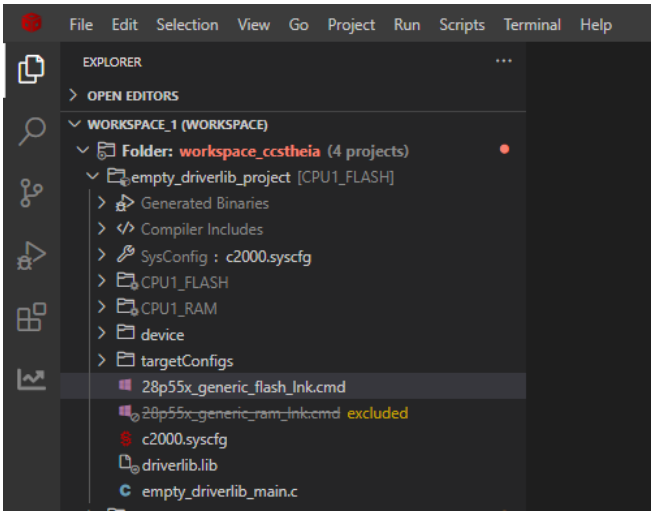
- 步骤 1：打开 CCS，单击 *File* 和 *Import Projects*。



- 步骤 2：从 C2000Ware SDK 导入 *empty_project*。

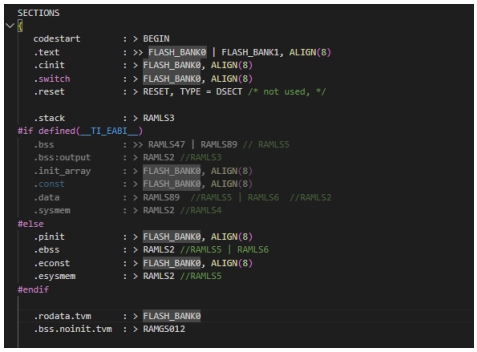


- 步骤 3：在导入的工程中打开 *f28p55x_generic_flash_lnk.cmd* 文件。

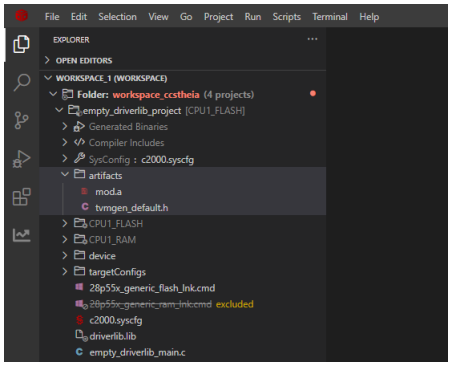


版权所有 © 2026, Texas Instruments Incorporated

- 步骤 4：将 .rodata.tvm 段添加到 Flash，并将 .bss.noinit.tvm 段放置在全局共享 SRAM 中，以便 NPU 硬件访问。例如，可使用 F28P55x MCU 器件上的 RAMGS0、RAMGS1、RAMGS2 或 RAMGS3。

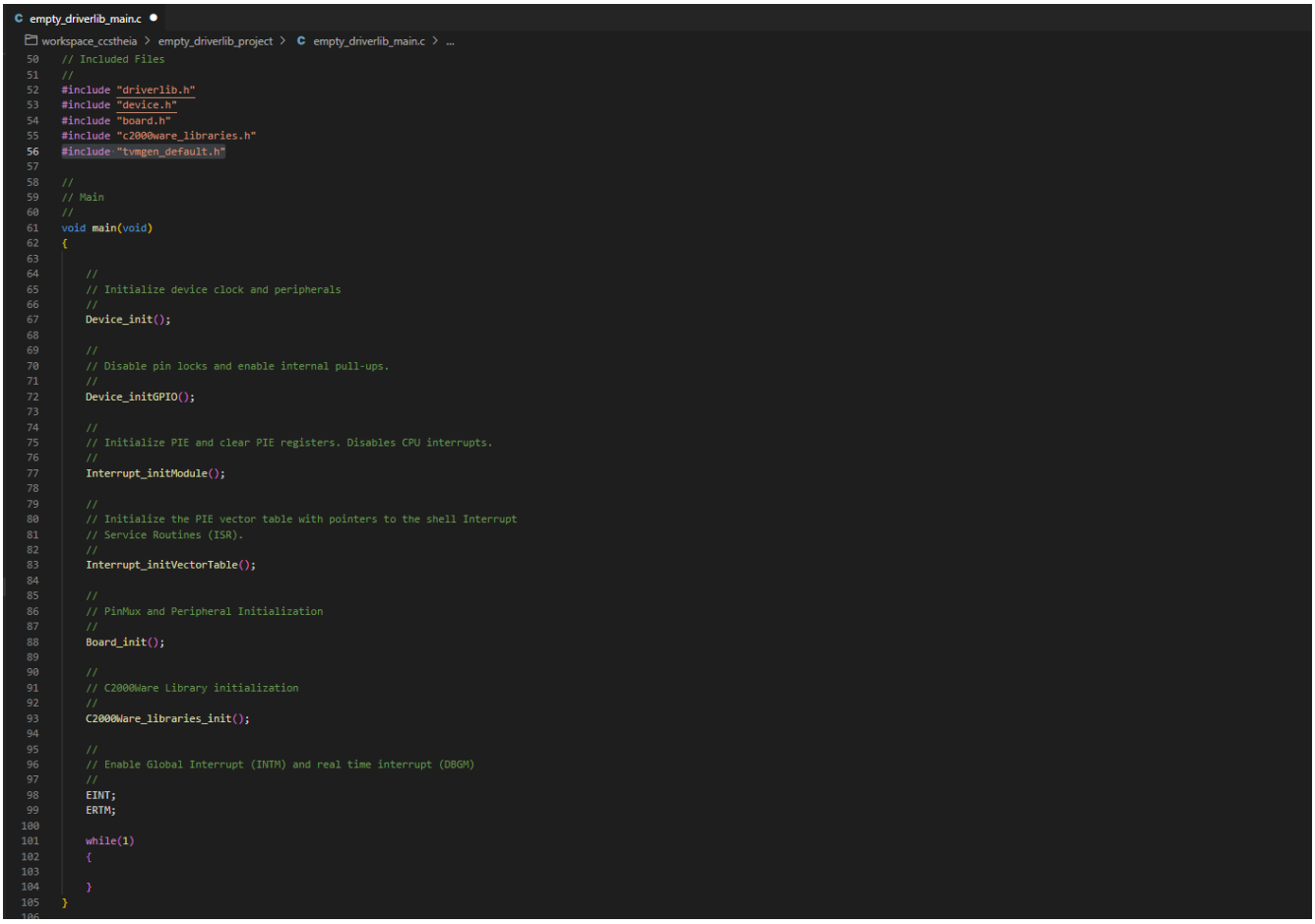


- 步骤 5：在工程中创建名为 ***artifacts*** 的文件夹，并将编译后的库文件和头文件放入该文件夹。



- 步骤 6：在应用程序中包含该头文件，然后编写应用程序以使用运行在 NPU 上的机器学习模型。下一节将详细介绍 NPU 接口。

版权所有 © 2026, Texas Instruments Incorporated



9.2 了解 NPU 接口

头文件（tvmgen_default.h）提供关键结构和函数，作为应用程序与 NPU 之间的接口。要成功集成神经网络，必须了解这些组件。

9.2.1 Key Interface Components

该头文件定义了若干重要元素：

- *Input and Output Structures*：这些结构提供了在神经网络与应用程序之间传递数据的机制。

```
struct tvmgen_default_inputs {     void* input;  // Points to input data (float for sine case) };str
uct tvmgen_default_outputs {     void* output;  // Points to output buffer for results };
```

Code 11: Input and Output structures generated during the compilation process

- *Initialization Function:*应用程序启动时必须调用此函数，以配置 NPU 硬件。

```
void TI_NPU_init();
```

Code 12: NPU initialization API

版权所有 © 2026, Texas Instruments Incorporated

- *Execution Function:*此函数会触发 NPU 执行神经网络。

```
  int32_t tvmgen_default_run(
    struct tvmgen_default_inputs* inputs,
    struct tvmgen_default_outputs* outputs
);
```

Code 13: API to trigger NPU

- *Completion Flag:*此标志表示神经网络处理是否已完成。

```
extern volatile uint8_t tvmgen_default_finished;
```

Code 14: Flag to denote end of NPU processing

9.2.2 Basic Usage Pattern

使用 NPU 的典型工作流程如下：

- *One-time Initialization:*系统启动期间调用 TI_NPU_init()。
- *Input Preparation:*创建并填充输入结构，将其指针指向输入数据。
- *Output Allocation:*创建输出结构，将其指针指向用于接收结果的变量。
- *Model Execution:*使用输入和输出结构调用 tvmgen_default_run()。
- *Completion Monitoring:*监测 tvmgen_default_finished 标志，以确定处理何时完成。
- *Result Utilization:*完成标志置位后，输出数据即可供应用程序使用。

版权所有 © 2026, Texas Instruments Incorporated

10 在嵌入式环境中测试模型

模型训练和转换成功后，还必须直接在 F28P55x 微控制器上全面测试正弦函数近似模型，以验证其在实际部署环境中的性能。本节介绍在目标硬件上运行神经网络模型时采用的测试方法及所得结果。

10.1 可视化性能评估

测试过程的一个关键环节，是直观比较预测正弦值与实际正弦值。我们使用 CCS 绘图工具，在整个输入范围内绘制神经网络预测结果和真实正弦函数曲线。

图 10-1 展示了 NPU 生成的正弦近似值与真实正弦函数值之间的可视化对比：

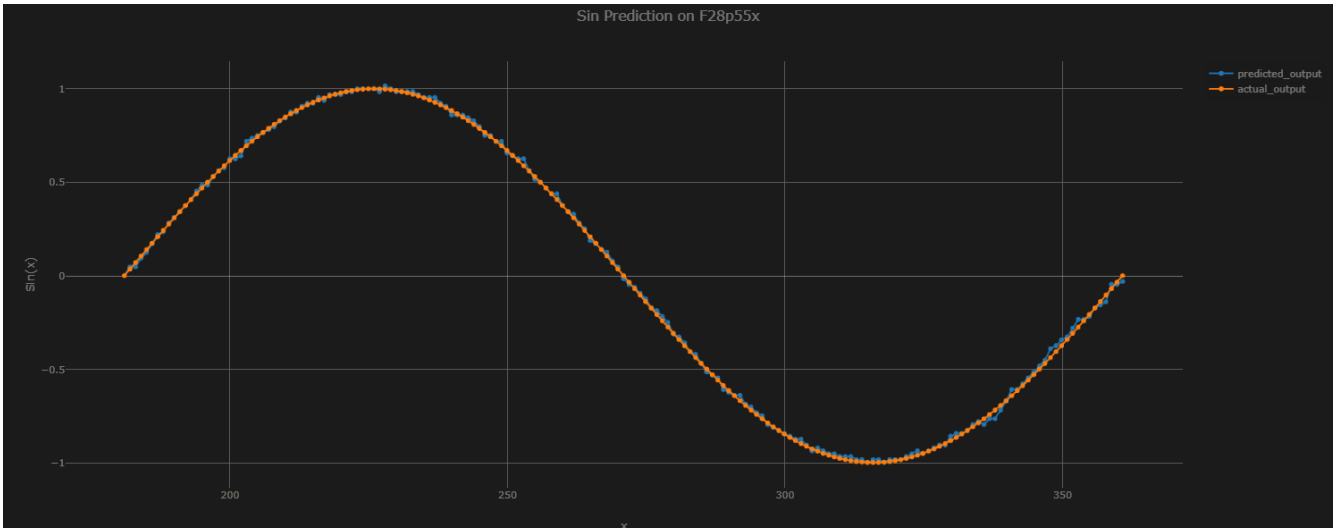


图 10-1. 神经网络 正弦函数近似模型在 F28P55x 上的性能

可视化结果展现了模型在实际硬件上的若干关键性能：

- *Consistent Performance:*模型在整个波形范围内均保持较高准确率，包括波峰、波谷等极值点和过零点等关键位置。
- *Smooth Transitions:*尽管 NPU 采用量化运算，近似曲线仍保持平滑过渡，没有可见的不连续或伪影。
- *Complete Cycle Coverage:*该测试覆盖正弦函数从 0 到 360 度的完整周期，确认波形在各相位均能保持稳定性能。

可视化结果有力地证明，该模型成功从训练环境迁移到 F28P55x 硬件，准确率仅有轻微下降。预测值与实际值几乎完全重合，表明量化感知训练方法能够在编译过程中有效保持模型保真度。

10.2 定量性能指标

除目视检查外，我们还计算了全面的定量指标，以客观评估模型在 F28P55x NPU 硬件上的性能。定量评估得到以下关键性能指标：

- *Mean Absolute Error (MAE):*0.01214
- *Maximum Error:*0.0973
- *Latency (ms):*0.214
- *R^2 Score:*0.9997

版权所有 © 2026, Texas Instruments Incorporated

这些指标全面评估了模型的准确率和性能。误差指标量化了模型近似结果的精度，计时数据则体现了 NPU 硬件相较主 CPU 软件执行的效率优势。准确率和速度兼备，可支持汽车和工业控制系统中对时序和精度要求严格的实时应用。

版权所有 © 2026, Texas Instruments Incorporated

11 NPU 在实时信号链中的集成

使用预定义数据集进行初步模型验证可以提供有价值的性能信息，但 NPU 应用的真正考验在于能否处理实时数据。本节将介绍完整的实时信号处理链路：通过 F28P55x NPU 上的神经网络推理，将实时模拟输入转换为有用的输出。

11.1 应用框图

图 11-1 展示了为验证 F28P55x 微控制器上的神经网络处理单元（NPU）实现而搭建的仿真环境。该架构图展示了从信号生成、处理到可视化的完整流程，说明神经网络模型如何实时将锯齿波转换为正弦波。

- *Signal Generation:*图中最左侧为配置成 DAC 的 CMPSS 模块，用于生成幅度范围为 0-360 个单位的锯齿波。上方方框内的折线图形表示该波形，它作为系统的输入激励。
- *Signal Acquisition:*锯齿波输入 ADC（模数转换器）模块，由其将模拟信号转换为数字信号。输出仍保持相同的数值范围（0-360），但已变为数字域数据。
- *Neural Network Processing:*图中央是 F28P55x 微控制器，内部包含 NPU 和正弦模型。输入值（标记为“x”）传入正弦模型，由模型计算相应的正弦值。
- *Output Generation:*神经网络输出的范围为 -1 到 1（正弦值的自然取值范围），该输出随后送至第二个 DAC，转换回模拟域。
- *Visualization:*图中最右侧显示最终输出，并由示波器呈现平滑的正弦波形，直观验证了从锯齿波到正弦波形的成功转换。

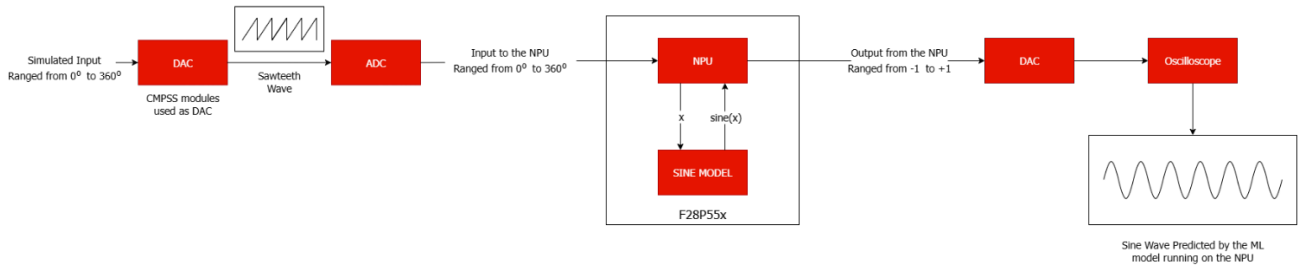


图 11-1. 应用端数据流

11.2 应用程序代码实现

以下代码展示如何实现完整信号处理流程，集成 CMPSS、ADC、NPU 和 DAC 组件，并使用神经网络模型将锯齿波转换为正弦波：

```
while(1){
   for(i = 0; i < MAX_ADC_VALUE; i++)
   {
      // Set the CMPSS low DAC value to generate sawtooth waveform
      CMPSS_setDACValueLow(CMPSS1_BASE, i);
      // Trigger ADC conversion to read back the analog signal
      ADC_forceSOC(myADC0_BASE, myADC0_SOC0);
      // Read the result from the ADC
      adcResult = ADC_readResult(myADC0_RESULT_BASE, myADC0_SOC0);
      // Scale ADC value to phase angle in range [0, 2π)
      input = (float)adcResult * (2.0f * M_PI / (MAX_ADC_VALUE + 1));
      // Prepare input/output structures for the neural network
      struct tvmgen_default_inputs inputs = { (void*)input_arr };
      struct tvmgen_default_outputs outputs = { output_arr };
      // Execute the neural network model to predict sine value
      tvmgen_default_run(&inputs, &outputs);
      // For NPU mode, wait until the neural network processing is complete
      #if defined(TVMGEN_DEFAULT_TI_NPU)
```


版权所有 © 2026, Texas Instruments Incorporated

```
while (!tvmgen_default_finished);      #endif      // Scale the sine output from [-1, 1] to DAC range [0, 40
95]      dacValue = (uint16_t)((output + 1.0f) * (MAX_DAC_VALUE / 2.0f));      // Output the predicte
d sine wave to the DAC      DAC_setShadowValue(myDAC0_BASE, dacValue);      // Delay to control
the wave frequency      DEVICE_DELAY_US(WAVE_DELAY_US);  // Delay between samples    } }
```

Code 15: Main Application Code for Real-Time Neural Network-Based Sine Wave Generation on F28P55x

11.3 使用的硬件组件

该仿真环境使用了 F28P55x 平台的多项关键硬件组件：

- *CMPSS DAC (Comparator Subsystem):*CMPSS 通常用于比较器功能，在此被重新用作信号发生器。CMPSS 模块包含内部 12-bit DAC，通过程序递增其输出以生成锯齿波。这种方法展示了 F28P55x 外设的灵活性，无需外部硬件即可生成测试信号。
- *ADC Module:*F28P55x 集成的 12-bit 模数转换器（ADC）用于对生成的波形进行采样。
  实际应用中通常将其连接到外部传感器或信号源；但在此仿真环境中，ADC 对内部生成的波形进行采样，从而形成完整信号通路。
- *NPU Hardware Accelerator:*F28P55x 内部的专用神经网络处理单元（NPU）执行正弦模型计算，其性能和效率高于单独使用主 CPU。图中显示了 NPU 如何集成到处理链路中：接收数字输入并生成计算结果。图中的“Sine Model”表示已训练、编译并部署到 NPU 的神经网络模型，其中包含将输入值转换为相应正弦值所需的权重和结构。
- *Buffered DAC:*独立的 DAC 通道将神经网络的浮点输出转换回模拟信号，以供观察。该 DAC 与 CMPSS DAC 在典型用法和性能特性方面有所不同。

11.4 硬件验证结果

图 11-2 所示的示波器截图为整个 NPU 应用提供了明确验证，展示了神经网络推理如何在 F28P55x 平台上实时将锯齿波转换为正弦波。

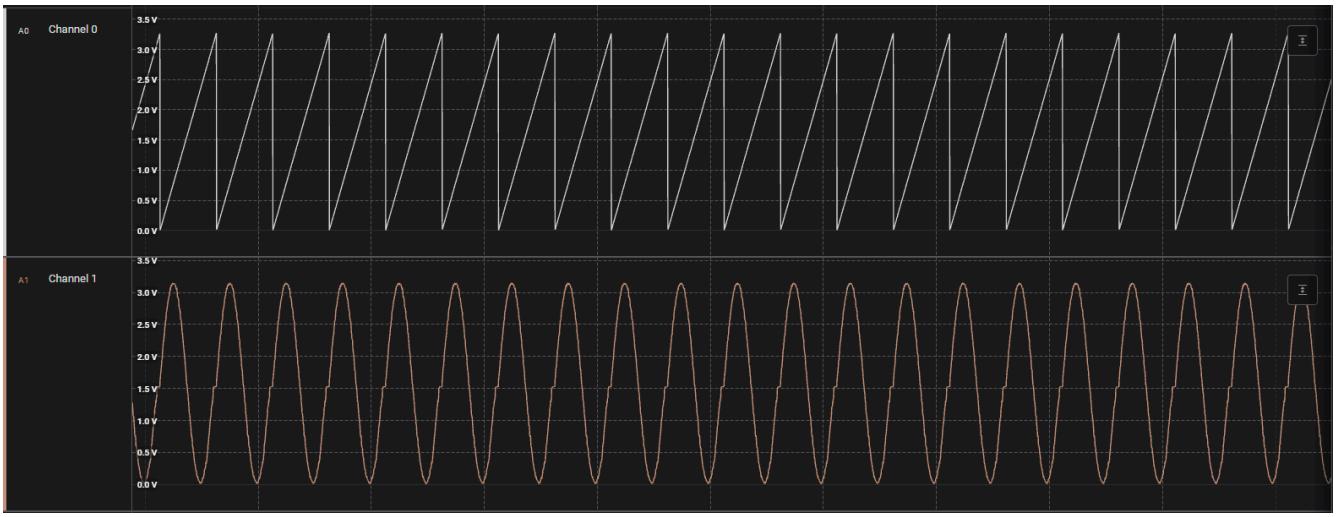


图 11-2. 示波器上的应用验证

11.4.1 Input Signal Characteristics

顶部波形（通道 0）显示 CMPSS DAC 生成的锯齿波：

版权所有 © 2026, Texas Instruments Incorporated

- 从 0V 到 3.3V 的线性斜坡清晰，复位边沿陡峭。
- 所有周期的周期和幅度均一致，表明信号生成稳定。
- 复位点处的陡变为神经网络带来了较大的测试挑战，因为输入函数在这些位置存在不连续点。

该线性斜坡输入是理想的测试信号，因为它会按规律遍历神经网络的输入范围，并在每个周期覆盖模型的完整工作范围。

11.4.2 Neural Network Output Analysis

底部波形（通道 1）显示神经网络生成的正弦波，并输出至 DAC：

- 正弦波形平滑，幅度范围为 0V 至 3.3V。
- 所有可见周期的幅度和频率均保持一致。
- 波峰和波谷处的失真极小，表明模型在关键位置具有出色的近似效果。
- 与输入锯齿波保持正确的相位关系，每个锯齿波周期都恰好对应一个正弦波周期。

- 部分过零点处存在轻微可见的不完美，这是由于 CMP1_DACL 的线性度限制；该信号源实际仅达到 10-bit 精度。

正弦波输出质量出色，波形平滑、形状完整，与理论正弦函数高度吻合。多个周期之间的一致性表明神经网络在连续运行时能够稳定、重复地产生可靠结果。

版权所有 © 2026, Texas Instruments Incorporated

12 关键设计决策及其影响

12.1 NPU 对数值的处理

f28p55x NPU 的设计基础是整数处理，因此在处理负值和浮点数据时会面临挑战。不过，通过适当调整神经网络架构和编译流程，NPU 也能有效处理这些数据类型。本文介绍了解决这些限制的完整设计方法。

12.1.1 Integer-Only Architecture

- NPU 硬件仅针对整数运算进行了优化。
- 仅原生支持无符号整数运算。
- 这一核心设计选择提升了嵌入式应用的处理效率，但要处理其他数据类型，就需要采用特殊方法。

12.1.2 Working with Negative and Floating-Point Values

- *Quantization Process:*负值和浮点数据都需要量化，映射到 NPU 的整数表示形式。
- *Automatic Conversion Layers:*编译过程会自动为输入插入量化层，并为输出插入反量化层。
- *Configuration Requirements:*必须明确配置编译流程，以保留数值的完整取值范围：

```
output_int: False  # Enables float output from integer NPU processing
```

Code 16: Flag to enable dequantized float output

- *Special Compiler Flags:*对于需要浮点或负值输出的应用（例如本例中的正弦函数），还需要设置其他编译标志：

```
skip_normalize=false output_int=false
```

Code 17: Flag to disable normalization

- *Processing Overhead:*这些额外的量化/反量化层会带来少量 计算开销。

12.2 支持的神经网络层及限制

F28P55x NPU 支持特定类型的神经网络层，并对其施加相应约束。了解这些能力对于设计可在该硬件上成功编译和部署的模型至关重要。

12.2.1 Supported Layer Types

12.2.1.1 卷积层

- *First Convolution (FCONV):*支持单通道输入特征图（不支持深度卷积和逐点卷积）。
- *Generic Convolution (GCONV):*多通道输入数量必须是 4 的倍数（不支持深度卷积和逐点卷积）。
- *Depth-Wise Convolution (DWCONV):*对各个输入通道分别应用滤波器。
- *Point-Wise Convolution (PWCONV):*实现 1×1 卷积，以进行通道混合。
- *Point-Wise Convolution with Residual (PWCONVRES):*包含用于残差学习的跳跃连接。
- *Transposed Convolution (TCONV):*支持上采样运算。

12.2.1.2 其他核心层

- *Fully-Connected (FC):*支持全连接/线性运算。
- *Average Pooling (AVGPOOL):*通过计算平均值执行下采样。
- *Max Pooling (MAXPOOL):*通过选择最大值执行下采样。

12.2.1.3 灵活性

- 输入、输出和残差张量数据（特征图）均为 8-bit 整数，可采用有符号或无符号形式。
- 权重可以为 2、4 或 8-bit，并且始终为有符号数。

版权所有 © 2026, Texas Instruments Incorporated

- 未来将支持更多 4-bit 数据与 4-bit 权重的组合。
- 推理时仅支持批量大小为 1。
- 除 DWCONV 层外，卷积层的分组数始终为 1。未来将支持分组卷积。

- 除 FCONV 的输入通道外，输入/输出通道数必须是 4 的倍数。
- TCONV 层的步长必须与卷积核大小相同。
- 模型中的层数没有限制。
- 某一层的输入与输出可以具有不同的有符号性和位宽。
- 各层可以采用混合精度。例如，一层可使用 8-bit 权重，而另一层使用 2-bit 权重。

12.3 模型复杂度和大小限制

实验表明，在 F28P55x NPU 上运行时，模型复杂度受到明显限制。开发适用于该平台的神经网络应用时，必须了解这些限制。

12.3.1 Memory Constraints and Model Size

- *PC versus Embedded Development:*早期模型每层包含 1024 或 512 个神经元，模型大小为 2.5-4.2MB。这些模型可以在 PC 上成功运行，但完全无法装入 F28P55x。
- *Successful Architecture:*最终，我们选择了每个隐藏层仅包含 64 个神经元的小型模型，以平衡准确率和性能：– *Input layer:* 1×64 个权重 + 64 个偏置 = 128 个参数 – *Hidden layer:* 64×64 个权重 + 64 个偏置 = 4,160 个参数 – *Output layer:* 64×1 个权重 + 1 个偏置 = 65 个参数 – *Total:* 共 4,353 个参数（远少于大型架构）

- *Physical Constraints:*与所有边缘计算器件一样，F28P55x 受限于片上资源，因此在设计该平台的神经网络时，优化模型大小至关重要。

12.3.2 Optimization Process and Performance Trade-offs

- *Progressive Size Reduction:*我们系统地测试了不同规模的模型，发现每层最多包含 128 个神经元的模型可以装入器件，但每层 64 个神经元的配置在准确率和延迟之间最为均衡。

- *Memory versus. Accuracy:*存储器限制成为首要设计因素，因此我们必须从硬件限制出发反向设计，而不能只从准确率目标出发。
- *Implementation Considerations:*最终模型不仅能够装入器件，还具备以下优点：– 可使用 NPU 工具链可靠编译。– 正弦波近似精度足够。– 推理时间稳定，低于 1ms。

版权所有 © 2026, Texas Instruments Incorporated

13 基准测试

为量化在 F28P55x NPU 上部署神经网络所涉及的性能权衡，我们针对不同模型配置、部署平台和优化方法进行了全面基准测试。测试结果有助于了解为嵌入式应用选择模型架构时需要考虑的实际因素。

13.1 模型性能对比

性能评估采用以下关键指标：

- *Latency (ms):*处理单次推理所需的时间，以毫秒为单位。数值越低，响应速度越快，对实时应用尤其重要。
- *Throughput (samples/sec):*每秒可处理的推理次数。数值越高，处理能力越强，对流式数据应用尤其重要。
- *Mean Absolute Error (MAE):*预测值与实际值之差的绝对值平均数。数值越低，预测准确率越高。
- *R² Score:*决定系数，用于衡量模型对数据的拟合程度。数值越接近 1.0，预测性能越好；1.0 表示预测完全准确。
- *Maximum Error:*任意预测值与对应实际值之间的最大绝对误差。数值越低，最坏情况下的性能越好。

共评估了 3 种模型变体，分别针对不同神经元配置：

- *Reference Python Model:*仅可在 PC 上运行的标准实现。
- *Quantization-Aware ONNX Model:*采用量化感知方法训练，并在 PC 上以 ONNX 模型形式验证。
- *Deployed f28p55x Model:*将 ONNX 模型编译并部署到 F28P55x 硬件上。

13.1.1 128 - Neuron Model

表 13-1. Sine_128_Model 基准测试结果

Metrics	F28p55x CPU	F28p55x NPU
Latency [ms]	1.012	0.7116
Samples/sec	987	1405
MAE	0.0015	0.0097
R2 Score	0.9999	0.9996
Max Error	0.01583	0.04407

13.1.2 64 - Neuron Model

表 13-2. Sine_64_Model 基准测试结果

Metric	F28p55x CPU	F28p55x NPU
Latency [ms]	0.2706	0.2146
Samples/sec	3695	4659
MAE	0.017525	0.012144
R2 Score	0.997	0.9993
Max Error	0.19085	0.0979

13.1.3 16 - Neuron Model

表 13-3. Sine_16_Model 基准测试结果

Metric	F28p55x CPU	F28p55x NPU
Latency [ms]	0.0223	0.0252
Samples/sec	44643	39557
MAE	0.1588	0.02029
R2 Score	0.88812	0.8451
Max Error	0.86379	0.9618

版权所有 © 2026, Texas Instruments Incorporated

13.1.4 参考基准测试

为建立准确率基准，我们还在 PC 上测试了一个大型参考模型：

表 13-4. 1024 神经元模型的基准测试结果（参考模型）

指标	Python 参考模型（1024 个神经元）
延迟 [ms]	0.149
样本/秒	7188
MAE	0.002171
R^2 分数	1
最大误差	0.0054

该参考模型准确率接近完美，但其大小达到 3.5-4.2MB，远超 F28P55x 的存储器容量。

13.2 性能分析

性能评估采用多项关键指标，量化模型在不同配置下的表现：

13.2.1 模型选择的权衡

128 神经元模型代表了该应用中准确率最高的配置。该模型准确度很高，在 NPU 上运行时 R^2 分数达到 0.9996，平均绝对误差（MAE）仅为 0.0097。但这一准确率需要付出明显的处理速度代价：延迟为 0.7116ms，吞吐量仅为 1,405 samples/sec。

64 神经元模型在准确率和硬件限制之间取得了平衡。模型仍保持较高准确率（R^2>0.99），同时显著提高了处理速度。在 NPU 上运行时，该模型可达到 4,659 samples/sec，吞吐量超过 128 神经元配置的 3 倍，而预测质量仅略有下降。

更小的模型能够大幅提高吞吐量，但准确率会显著下降。16 神经元模型在 NPU 上的执行速度为 39,557 samples/sec，是 128 神经元模型的 28 倍；但其 R^2 分数降至 0.8451，预测质量明显下降，对许多精度要求严格的应用而言无法接受。

13.2.2 CPU 与 NPU 性能对比

CPU 与 NPU 的性能对比可为实现方案选择提供参考：

复杂模型中的 NPU 优势
对于较大的模型，NPU 可显著提升性能。128 神经元模型在 NPU 上的运行速度比 CPU 快 29.7%（延迟分别为 0.7116ms 和 1.012ms），而 64 神经元模型的延迟降低了 20.7%。这一优势源于 NPU 专为并行神经网络计算设计的架构。

简单模型中的 CPU 优势
对于 16 神经元配置等非常小的模型，CPU 实际上比 NPU 更快。CPU 吞吐量为 44,643 samples/sec，高于 NPU 的 39,557 samples/sec，性能优势为 12.9%。这一反直觉结果源于向 NPU 传输数据产生的开销。对于 16 神经元模型，计算量很小，CPU 可直接在本机执行环境中处理，无需经历多次数据传输。由于模型规模很小，CPU 可在单一执行上下文中完成整个推理，避免 NPU 所需的存储器传输开销。每次 NPU 推理都需要设置 DMA
传输、配置加速器、等待处理完成并取回结果；这些操作耗时总和超过轻量模型本身的神经网络计算时间。换言之，模型很小时，使用专用硬件的“成本”会高于其计算收益。

版权所有 © 2026, Texas Instruments Incorporated

13.3 流水线各阶段的计时测量

我们使用 F28P55x 上的硬件定时器测量了信号处理流水线各阶段的耗时，并通过精密测量了解各组件对系统总延迟的影响。

表 13-5. 不同组件的流水线阶段计时

Pipeline Stage	Timing (ms)
Software Write Delay for CMPSS DAC Module	0.0006
Hardware Delay for ADC Conversion	0.00084
NPU Processing (64-neuron model)	0.21
Software Write Delay for Buffer DAC	0.000593
Total Pipeline Latency	0.212

这些精确测量结果表明：

- 外设操作：CMPSS 模块、ADC 转换和 DAC 输出速度极快，每项操作耗时均少于 1 微秒。
- NPU 主导：神经网络推理耗时占据流水线的绝大部分，占总延迟的 99.04%。使用 64 神经元模型时，NPU 处理时间约为流水线其他所有组件总耗时的 210 倍。
- 固定开销：所有外设操作（CMPSS、ADC、DAC）的总耗时仅为 0.002033ms，占流水线总延迟的比例不到 1%。

版权所有 © 2026, Texas Instruments Incorporated

14 总结

F28P55x 神经网络处理单元（NPU）显著提升了汽车和工业应用中的嵌入式机器学习能力，使器件可在本地执行推理，同时保持这些领域所需的确定性性能。本指南通过一个实用的正弦函数近似示例，全面介绍了 NPU 的能力、限制和实现方法。

14.1 主要能力与限制

NPU 可通过硬件加速执行神经网络，其性能显著优于主 CPU 上的软件实现；对于较大型模型，性能可提升 20-30 %。这种基于整数运算的计算引擎能够实现实时处理和确定性执行，同时与其他 C2000 外设保持无缝

集成。

不过，这些能力也受到重要因素的限制，包括存储器有限导致模型复杂度受限、架构更适合某些特定网络拓扑，以及量化带来的精度权衡。基准测试发现，NPU 在较大型模型上表现出色；对于非常小的神经网络，

例如只有 16 个神经元的模型，由于
需要与 NPU 进行数据传输而产生的开销，CPU 实际上可能运行得更高效。

14.2 开发工作流程

实现过程遵循结构化工作流，涵盖模型开发、编译和应用集成。首先通过量化感知训练，使模型适应 NPU 的纯整数运算；随后使用 TI 神经网络编译器生成硬件兼容产物；最后将这些组件集成到 CCS 工程中，并完成适当的外设配置，从而形成完整的信号处理流水线。

14.3 模型设计注意事项

通过系统化试验，本指南表明，成功实现 NPU 应用需要谨慎设计架构，在准确率要求与硬件限制之间取得平衡。正弦函数示例揭示了模型规模方面的重要结论：

- 最优的 64 神经元架构在保持出色准确率（$R^2 > 0.99$）的同时，也满足存储器 限制。
  。
- 较大型模型（128 个神经元）准确率略高，但吞吐量显著降低，不过相比 CPU 执行仍能受益于 NPU 加速。
- 较小模型（8-16 个神经元）吞吐量更高，但准确率代价较大。

14.4 实现挑战与解决方案

本指南介绍了 NPU 实现过程中遇到的若干实际挑战：

- *Negative and Floating-Point Values:*使用正确的反量化方法和编译设置，处理范围在 [-1, 1] 内的正弦值。
- *Neural Network Layer Support:*设计模型时利用受支持的层类型，同时避免使用不受支持的运算。
- *Memory Limitations:*系统化地缩小模型规模，使其符合硬件限制。

14.5 更广泛的应用

虽然本指南以正弦函数近似模型为例，但其中的技术和方法也适用于多种汽车和工业应用，包括：

- 通过振动或声学信号分析实现预测性维护。
- 传感器数据流中的异常检测。
- 基于神经网络建模的先进控制系统。
- 通过传感器融合提升感知和决策能力。

版权所有 © 2026, Texas Instruments Incorporated

F28P55x NPU 将机器学习能力直接带入嵌入式控制系统，使智能应用能够在边缘端独立运行，同时满足汽车和工业环境对可靠性的要求。神经网络直接运行在微控制器上，系统无需向外传输数据，即可在本地作出复杂决策。这种方式能保持响应时间可预测（对安全系统至关重要），同时比传统方案功耗更低。即使在云连接不可靠的严苛环境中，预测性维护、传感器融合和异常检测等应用也因此变得可行。NPU 在先进能力与关键控制系统的严格运行要求之间取得平衡，使系统能够增加智能处理能力，同时保留应用所需的确定性行为。

版权所有 © 2026, Texas Instruments Incorporated

15 参考文献

- Texas Instruments，《TMS320F28P55x 实时微控制器数据手册》，PDF
- Texas Instruments，EdgeAI Studio，网页
- Texas Instruments，TinyML-TensorLab，代码库
- Texas Instruments，《TI MCU 神经网络编译器用户指南》，网页
- Texas Instruments，C2000 MCU 的 C2000Ware，C2000 SDK，网页
- Texas Instruments，F28P55X LaunchPad 开发套件，产品详情网页

版权所有 © 2026, Texas Instruments Incorporated

重要声明和免责声明

TI 按“现状”提供技术和可靠性数据（包括数据表）、设计资源（包括参考设计）、应用或其他设计建议、Web 工具、安全信息及其他资源，并且不承担任何缺陷责任。TI 不作任何明示或暗示的保证，包括但不限于适销性、特定用途适用性或不侵犯第三方知识产权的暗示保证。

这些资源面向使用 TI 产品进行设计的专业开发人员。您需独自负责：(1) 为您的应用选择合适的
TI 产品；(2) 设计、验证并测试您的应用；以及 (3) 确保您的应用符合适用标准及其他安全、安保、监管或其他要求。

这些资源如有变更，恕不另行通知。TI 仅允许您将这些资源用于开发使用本文所述 TI 产品的应用。禁止以其他方式复制或展示这些资源。TI 不授予您使用任何其他 TI 知识产权或第三方知识产权的许可。对于因您使用这些资源而产生的任何索赔、损害、成本、损失和责任，TI 不承担责任，您应全额
赔偿 TI 及其代表，并使其免受相关损失。

TI 产品受 TI 销售条款、TI 通用质量指南或其他适用条款约束；相关条款可在 ti.com 上查阅，或随 TI 产品一同提供。TI 提供这些资源不会扩大或以其他方式改变 TI 产品适用的保证或免责声明。除非 TI 明确将产品指定为定制产品或客户指定产品，否则 TI 产品均为标准目录通用器件。

TI 反对并拒绝您可能提出的任何附加或不同条款。

重要声明

版权所有 © 2026, Texas Instruments Incorporated

最后更新 2025/10